

How do I build things?

Computer Psycho Union
University of Nottingham Ningbo China

VS Code • Terminal • Git & GitHub — your first toolchain

Lesson 0 • Session 04 — CPU Tech Group

5 October 2026

LESSON 0

TOOLCHAINS

The plan for today



01 **The need** week one, in five scenes

02 **Terminal** the fifty-year-old interface

03 **VS Code** where you write code

04 **Git & GitHub** keep and share

05 **Markdown & the toolchain** how it fits together

2 / 32

06 **Homework** before next week

THE NEED

01

What week one asks of you

01 • Week one, in five scenes



- **Write a program** — tip calculator • due Friday
- **Make it run** — on the grader's machine too
- **Hand it in** — Moodle zip • later, a Git repo
- **Team project** — four people • one codebase
- **Read code** — tutorials → research

Not genius — tools and habits.

01 • Four needs, four tools



You need to	The tool	It gives you
Talk to the machine	Terminal	run • install • automate
Edit code	VS Code	highlighting • autocomplete • run
Keep and share	Git + GitHub	save points • teamwork
Write documents	Markdown	READMEs • reports

One afternoon of setup — every week until graduation.

TERMINAL

02

Where it all began

02 • Fifty years, one window



1969 • THE TELETYPE

- **Teletype** — keyboard + paper
- **One room-sized machine** — many users
- **Unix is born here** — everything is a command

1978 • THE GLASS TERMINAL

- **VT100** — paper becomes a screen
- **Shell** — the program reading your lines
- **Terminal.app** — this screen, emulated

1984 • GUI ARRIVES

- **Macintosh** — windows • icons • mouse
- **Point-and-click wins the home**

TODAY

02 • The black window, in two words



- **Terminal** — the window
- **Shell** — the program inside • zsh • bash • PowerShell
- **Every button is a command** underneath
- **You own several already** — macOS • Windows • VS Code (Ctrl+`)
- **Why type?** — chains • scripts • no screen needed



8 / 32
Terminal.app —
since 2001,
emulating 1978

02 • Five commands you already know



What you'd click in Windows	What you type	It does
open a folder to look inside	<code>ls</code>	list
double-click in / back out	<code>cd folder</code> • <code>cd ..</code>	move
right-click → New → Folder	<code>mkdir project</code>	create
right-click → Rename	<code>mv old.txt new.txt</code>	rename • move
right-click → Delete	<code>rm old.txt</code>	gone — no trash

macOS • Linux: identical. Copy: `cp a.txt b.txt`.

`rm` skips the Recycle Bin. Folders: `rm -r project`.

02 • One line, one new project



```
mkdir my-first-project && cd my-first-project && code .
```

- **Folder → in → open** — zero mouse
- `.` — "here"
- `&&` — next runs only if the last succeeded
- **The line grows** — `git init` • `git push` soon

02 • Where it will find you



- **Installing** — `pip install` • `brew install` • `winget install`
- **Autograders** — coursework runs from one
- **SSH** — lab servers • cloud VMs • no GUI
- **Automation** — 3am, nobody clicking

VS CODE

03

Where the code gets written

03 • Your workshop for four years



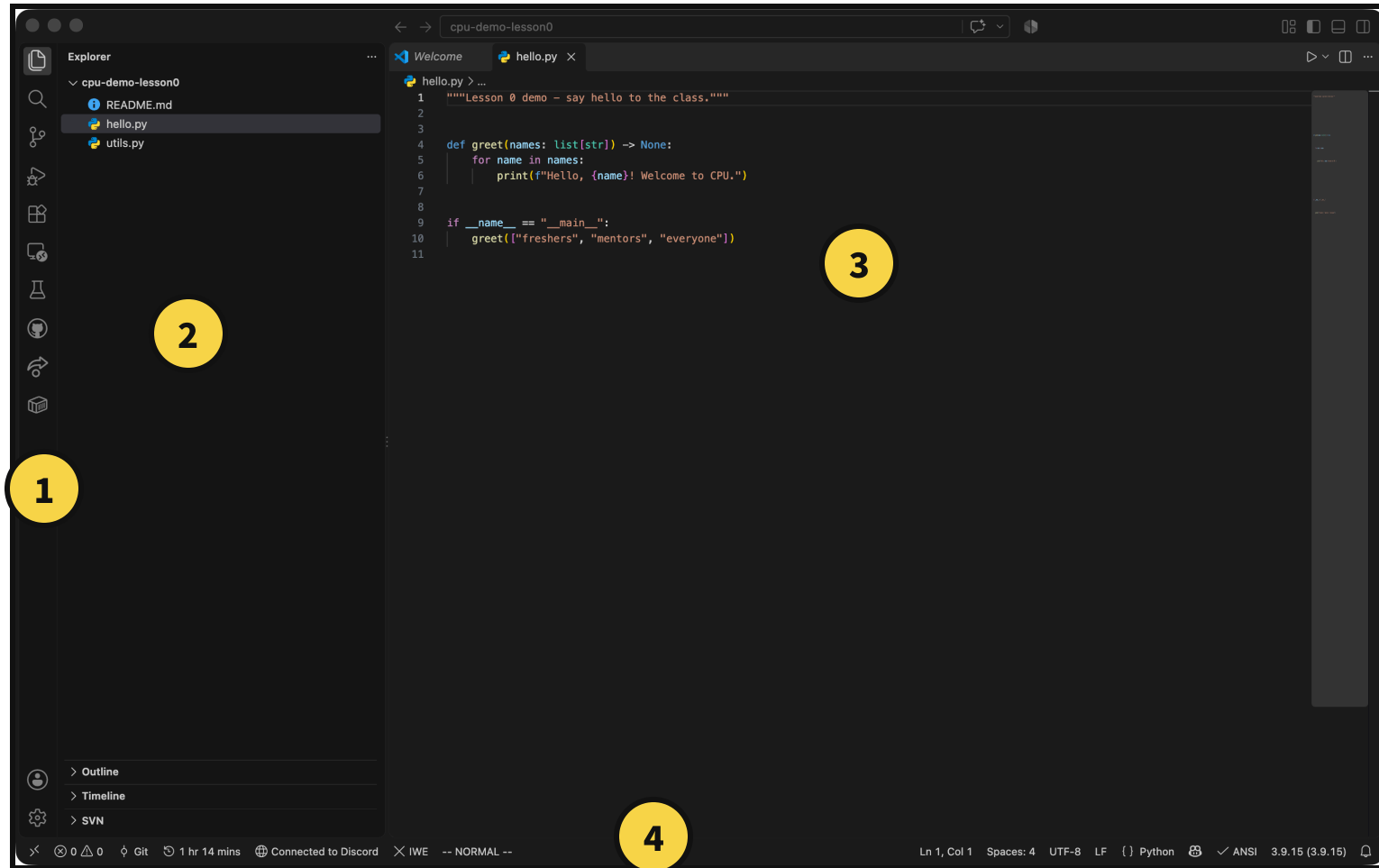
- **Not Word** — code needs a code editor
- **Free • everywhere • huge** — the world's default
- **Bare on purpose** — brains arrive as extensions
- **Today** — install → window → Python → done



13 / 32

free • macOS /
Windows / Linux

03 • One window, five areas



1 Activity Bar
files • search • git • extensions

2 Explorer
your project's files

3 Editor
tabs • code • minimap

4 Status Bar
branch • Ln/Col • language

5 Panel
terminal — Ctrl+` to open

14 / 32

Same five areas — first day to final project.

03 • Teaching the editor your language



- **Out of the box** — colour only
- **Step 1 — add the language** — Python extension
- **Step 2 — pick the interpreter** — the status-bar picker
- **Step 3 — Run** — output in the panel below
- **Same pattern forever** — C/C++ • Java • Rust

03 • Extensions worth installing today



Extension	Why
Python — Microsoft	run • debug • autocomplete
C/C++ — Microsoft	the spring C courses
Prettier	one format for the team
GitLens	who changed this line
Chinese (Simplified) Language Pack	menus in Chinese

Ctrl+Shift+X • all free • can't say why? Uninstall.

03 • Two shortcuts and one habit



- **Ctrl/Cmd + Shift + P** — Command Palette — every command
- **Ctrl/Cmd + P** — Quick Open — any file
- **Auto Save: afterDelay** — crashes stop eating homework

Twice with the mouse this week? It has a shortcut.

03 • Beyond VS Code



- **JetBrains** — PyCharm • IntelliJ • CLion — student licences
- **Vim** — on every server you'll ever meet
- **Cursor** — AI-first • VS Code base
- **Pick one — go deep**



18 / 32

PyCharm — the
Python one

LIVE DEMO

A

A folder, a file, and a run

VS Code — open a folder, write `hello.py`, run it, meet the built-in terminal

GIT & GITHUB

04

Keep everything, share everything

04 • A problem you already know



`essay.docx -> essay_v2.docx -> essay_final.docx -> essay_final_REAL.docx`

- **Git** — save points for a folder
- **GitHub** — the cloud home for Git folders
- **Three names** — Git the tool • GitHub the site • Desktop the driver
- **Coming for you** — team coursework • internships • research

04 • The loop, in one picture



- **commit** — labelled save point, on your laptop
- **push** — upload your save points
- **pull** — download everyone else's

edit • commit • push • pull — 90% of Git.

04 • GitHub Desktop — the loop, no typing



1. **Clone** — File → Clone repository
2. **Edit in VS Code** — Desktop lists every change
3. **Commit** — tick files • honest message • "Commit to main"
4. **Push origin / Pull origin** — yours up • theirs down

Hover any button — Desktop shows the git command underneath.



write
your laptop



commit • push
GitHub Desktop



see it online
github.com

04 • Branches — parallel timelines



- **Branch** — a parallel timeline
- **main stays runnable**
- **Try → merge or delete**
- **Teams** — own branch each • merge by pull request
- **In Desktop** — one dropdown: new • switch • merge • publish

Not week one. Week ten — file this page.

04 • More than a backup



- **Public workshop** — a profile beats a CV line
- **Open source** — issues • pull requests • reviews
- **Coursework & research** — team repos • reproducible code
- **Every repo opens on its README** — next section

LIVE DEMO

B

Edit to GitHub in four clicks

GitHub Desktop — clone, change, commit, push, then refresh the page and watch it land

26 / 32

05

Markdown, and how it all fits

05 • Markdown — write once, read anywhere



YOU TYPE

```
# My project
## Setup
- install **Python 3.12**
- run `python hello.py`
- see [the docs](https://...)
```

READERS SEE

My project

Setup

- install **Python 3.12**
- run `python hello.py`
- see [the docs](#)

- **Plain text, lightly formatted** — readable raw • pretty rendered
- **Everywhere** — GitHub • Jupyter • notes apps
- **This deck** — Markdown, rendered by Marp

05 • The toolchain so far



RUN

Terminal

run • install • automate



WRITE

VS Code

code • autocomplete •
extensions



VERSION

Git + GitHub

save points • shared in the
cloud



DOCUMENT

Markdown

READMEs • reports • this
deck



LATER THIS SERIES — SSH • DOCKER • CI • AI IN THE TERMINAL

One job each — together, a workshop.

This week's homework



INSTALL

- VS Code + the Python extension
- GitHub Desktop + a GitHub account

CREATE

- a folder with `hello.py` and `README.md`
- run it: `python hello.py`

PUSH

- repository in GitHub Desktop → commit → push
- link in the group chat

30 / 32

Bring a laptop next session.



Where to learn more

- code.visualstudio.com/docs — VS Code docs
- git-scm.com/book — Pro Git • chapters 1–3
- desktop.github.com — Desktop guides
- markdownguide.org — the cheat sheet
- github.com/CompPsyUnion — last year's notes

Stuck? Group chat • any mentor.

THANK YOU

04

How do I build things?